Physics-Informed Neural Operators (PINO)

1 Reference

Physics-Informed Neural Operators (PINO) combine operator learning with the physics constraints imposed by the governing partial differential equation (PDE).

A principal formulation is given in:

> Z. Li et al., "Physics-Informed Neural Operator for Learning Partial Differential Equations," *arXiv preprint*, 2021.

arXiv:2111.03794.

2 Motivation

A conventional neural operator learns an approximation of a solution operator

$$\mathcal{G} : a \longmapsto u, \tag{1}$$

where a denotes the input function, coefficients, forcing terms, initial conditions, or boundary conditions, and u is the solution of a PDE.

The training objective of an ordinary supervised neural operator is typically

$$\mathcal{L}_{\text{data}} = \frac{1}{N} \sum_{n=1}^{N} \left\| \mathcal{G}_\theta(a^{(n)}) - u^{(n)} \right\|^2. \tag{2}$$

This requires solution data.

PINO additionally imposes the governing physical equations,

$$\mathcal{F}\,[u; a] = 0. \tag{3}$$

Therefore the learned operator is constrained not only by observed solutions but also by the PDE itself.

3 General PDE Formulation

Consider a general PDE

$$\mathcal{F}\left(u(x,t), a(x,t), x, t\right) = 0, \qquad (x,t) \in \Omega \times [0,T]. \tag{4}$$

The PDE may be accompanied by boundary conditions

$$\mathcal{B}\,[u] = g, \qquad x \in \partial\Omega, \tag{5}$$

and initial conditions

$$\mathcal{I}\,[u] = u_0. \tag{6}$$

The exact solution operator is

$$u = \mathcal{G}(a). \tag{7}$$

The neural operator approximates this mapping by

$$\hat{u} = \mathcal{G}_\theta(a). \tag{8}$$

4 General Neural Operator

A general neural operator can be written as a composition

$$\boxed{\mathcal{G}_\theta = Q_\theta \circ \mathcal{L}_\theta^{(L)} \circ \cdots \circ \mathcal{L}_\theta^{(1)} \circ P_\theta}. \tag{9}$$

Here,

- P_θ is the lifting operator,
- $\mathcal{L}_\theta^{(l)}$ is the l-th neural operator layer,
- Q_θ is the output projection.

Examples include

$$\text{FNO}, \quad \text{DeepONet}, \quad \text{GNO}, \quad \text{Transolver}, \quad \text{GNOT}, \tag{10}$$

and other operator-learning architectures.
The important observation is that PINO does not require a particular operator architecture.

5 Core Idea of PINO

Given any differentiable neural operator

$$\hat{u}_\theta = \mathcal{G}_\theta(a), \tag{11}$$

evaluate the PDE residual on its prediction.
Define

$$\boxed{r_\theta = \mathcal{F}\left[\hat{u}_\theta; a\right].} \tag{12}$$

The physics loss is then

$$\mathcal{L}_{\text{PDE}} = \left\|r_\theta\right\|^2. \tag{13}$$

Boundary and initial conditions are imposed through additional residuals.
Thus,

$$\boxed{\mathcal{L}_{\text{PINO}} = \lambda_{\text{data}}\mathcal{L}_{\text{data}} + \lambda_{\text{PDE}}\mathcal{L}_{\text{PDE}} + \lambda_{\text{BC}}\mathcal{L}_{\text{BC}} + \lambda_{\text{IC}}\mathcal{L}_{\text{IC}}}. \tag{14}$$

6 How to Construct a PINO from Any Neural Operator

Suppose an existing neural operator has already been constructed,

$$\hat{u} = \mathcal{G}_\theta(a). \tag{15}$$

To convert it into a PINO, the architecture itself does not necessarily need to be replaced.
The construction consists of four steps.

6.1 Step 1: Train or Initialize the Neural Operator

Start with

$$\hat{u} = \mathcal{G}_\theta(a). \tag{16}$$

The model may be an FNO, DeepONet, GNO, Transolver, or another differentiable neural operator.

6.2 Step 2: Define the Governing PDE

Write the governing equation in residual form,

$$\mathcal{F}[u; a] = 0. \tag{17}$$

For example, consider the one-dimensional viscous Burgers equation,

$$u_t + uu_x - \nu u_{xx} = 0. \tag{18}$$

The PDE residual is

$$\boxed{r(x,t) = u_t + uu_x - \nu u_{xx}.} \tag{19}$$

6.3 Step 3: Evaluate the Residual on the Neural Operator Output

The neural operator produces

$$\hat{u}_\theta = \mathcal{G}_\theta(a). \tag{20}$$

Substitute this prediction into the PDE:

$$r_\theta = \mathcal{F}[\hat{u}_\theta; a]. \tag{21}$$

For Burgers' equation,

$$r_\theta = \frac{\partial \hat{u}_\theta}{\partial t} + \hat{u}_\theta \frac{\partial \hat{u}_\theta}{\partial x} - \nu \frac{\partial^2 \hat{u}_\theta}{\partial x^2}. \tag{22}$$

6.4 Step 4: Add the Physics Loss

The PDE loss is

$$\boxed{\mathcal{L}_{\text{PDE}} = \frac{1}{N_c} \sum_{i=1}^{N_c} |r_\theta(x_i, t_i)|^2.} \tag{23}$$

The original data loss can then be combined with the physics loss.

7 Data Loss

Suppose the training dataset consists of

$$\mathcal{D} = \left\{ (a^{(n)}, u^{(n)}) \right\}_{n=1}^{N}. \tag{24}$$

The supervised loss is

$$\mathcal{L}_{\text{data}} = \frac{1}{N}\sum_{n=1}^{N}\left\|\mathcal{G}_\theta(a^{(n)}) - u^{(n)}\right\|_2^2. \tag{25}$$

PINO does not require this term to be the only source of supervision.

8 Boundary-Condition Loss

For a Dirichlet boundary condition,

$$u(x,t) = g(x,t), \qquad x \in \partial\Omega, \tag{26}$$

define

$$r_{\text{BC}} = \hat{u}_\theta - g. \tag{27}$$

The boundary loss is

$$\boxed{\mathcal{L}_{\text{BC}} = \frac{1}{N_{\text{BC}}}\sum_{i=1}^{N_{\text{BC}}}\left|\hat{u}_\theta(x_i,t_i) - g(x_i,t_i)\right|^2.} \tag{28}$$

9 Initial-Condition Loss

For

$$u(x,0) = u_0(x), \tag{29}$$

the initial-condition residual is

$$r_{\text{IC}} = \hat{u}_\theta(x,0) - u_0(x). \tag{30}$$

Therefore,

$$\boxed{\mathcal{L}_{\text{IC}} = \frac{1}{N_{\text{IC}}}\sum_{i}\left|\hat{u}_\theta(x_i,0) - u_0(x_i)\right|^2.} \tag{31}$$

10 Complete PINO Objective

The complete objective is

$$\boxed{\mathcal{L} = \lambda_d\mathcal{L}_{\text{data}} + \lambda_p\mathcal{L}_{\text{PDE}} + \lambda_b\mathcal{L}_{\text{BC}} + \lambda_i\mathcal{L}_{\text{IC}}.} \tag{32}$$

The parameters are optimized according to

$$\boxed{\theta^* = \arg\min_\theta \mathcal{L}(\theta).} \tag{33}$$

11 Automatic Differentiation

If the neural operator output is differentiable with respect to the coordinates, derivatives appearing in the PDE residual can be computed using automatic differentiation.

For example,

$$\hat{u}_x = \frac{\partial \hat{u}}{\partial x}, \tag{34}$$

and

$$\hat{u}_{xx} = \frac{\partial^2 \hat{u}}{\partial x^2}. \tag{35}$$

However, PINO implementations can also evaluate derivatives using spectral differentiation, finite differences, or other numerical differentiation techniques.

12 Spectral Physics Loss

For Fourier-based neural operators, derivatives can be evaluated in Fourier space.

Let

$$\hat{u}(x) = \sum_k \widehat{u}_k e^{ikx}. \tag{36}$$

Then

$$\widehat{\frac{\partial u}{\partial x}}_k = ik\widehat{u}_k, \tag{37}$$

and

$$\widehat{\frac{\partial^2 u}{\partial x^2}}_k = -k^2\widehat{u}_k. \tag{38}$$

Thus

$$u_x = \mathcal{F}^{-1}\left[ik\widehat{u}_k\right], \tag{39}$$

and

$$u_{xx} = \mathcal{F}^{-1}\left[-k^2\widehat{u}_k\right]. \tag{40}$$

This approach is particularly natural for periodic PDEs.

13 General Operator-Agnostic PINO Construction

The construction can be summarized mathematically as

$$\boxed{\text{Input} \xrightarrow{\mathcal{G}_\theta} \hat{u} \xrightarrow{\mathcal{F}} r_\theta}. \tag{41}$$

The physics loss is

$$\mathcal{L}_{\text{PDE}} = \|r_\theta\|^2. \tag{42}$$

Therefore any differentiable neural operator can be converted into a PINO by adding a differentiable physics-residual pathway.

The architecture remains

$$\boxed{\mathcal{G}_\theta = Q_\theta \circ \mathcal{L}_\theta^{(L)} \circ \cdots \circ \mathcal{L}_\theta^{(1)} \circ P_\theta}. \tag{43}$$

What changes is the training objective.

$$\boxed{\mathcal{L}_{\text{data}} \quad \longrightarrow \quad \mathcal{L}_{\text{data}} + \mathcal{L}_{\text{physics}}.} \tag{44}$$

14 Example: FNO → PINO-FNO

Suppose the original FNO is

$$\hat{u} = \text{FNO}_\theta(a). \tag{45}$$

The PDE is

$$\mathcal{F}[u; a] = 0. \tag{46}$$

Then

$$r_\theta = \mathcal{F}[\text{FNO}_\theta(a); a]. \tag{47}$$

The resulting PINO-FNO loss is

$$\boxed{\mathcal{L}_{\text{PINO-FNO}} = \lambda_d \left\|\text{FNO}_\theta(a) - u\right\|^2 + \lambda_p \left\|\mathcal{F}[\text{FNO}_\theta(a); a]\right\|^2.} \tag{48}$$

15 Example: DeepONet → PINO-DeepONet

For DeepONet,

$$\hat{u}(y) = \sum_{k=1}^{p} b_k(a) t_k(y). \tag{49}$$

The PDE residual becomes

$$r_\theta = \mathcal{F}[\hat{u}_\theta; a]. \tag{50}$$

Thus

$$\boxed{\mathcal{L}_{\text{PINO-DeepONet}} = \lambda_d \|\hat{u}_\theta - u\|^2 + \lambda_p \|\mathcal{F}[\hat{u}_\theta; a]\|^2.} \tag{51}$$

16 Example: Transolver → PINO-Transolver

For Transolver,

$$\hat{u} = \text{Transolver}_\theta(a, X). \tag{52}$$

The PDE residual is

$$r_\theta = \mathcal{F}[\text{Transolver}_\theta(a, X); a]. \tag{53}$$

The PINO loss is

$$\boxed{\mathcal{L} = \lambda_d \mathcal{L}_{\text{data}} + \lambda_p \mathcal{L}_{\text{PDE}} + \lambda_b \mathcal{L}_{\text{BC}}}. \tag{54}$$

Thus the same principle applies regardless of the underlying operator architecture.

17 PyTorch: Generic PINO Wrapper

```python

import torch
import torch.nn as nn


class PINO(nn.Module):

    def __init__(
        self,
        operator,
        physics_residual,
        lambda_data=1.0,
        lambda_pde=1.0,
        lambda_bc=1.0,
        lambda_ic=1.0
    ):
        super().__init__()

        self.operator = operator
        self.physics_residual = physics_residual

        self.lambda_data = lambda_data
        self.lambda_pde = lambda_pde
        self.lambda_bc = lambda_bc
        self.lambda_ic = lambda_ic

    def forward(
        self,
        *args,
        **kwargs
    ):

        return self.operator(
            *args,
            **kwargs
        )

    def loss(
        self,
        prediction,
        target,
        pde_loss,
        bc_loss=None,
        ic_loss=None
    ):

        data_loss = torch.mean(
            (prediction - target) ** 2
        )

        total = (
            self.lambda_data * data_loss
            +
            self.lambda_pde * pde_loss
        )

```

```
        if bc_loss is not None:

            total += (
                self.lambda_bc
                * bc_loss
            )

        if ic_loss is not None:

            total += (
                self.lambda_ic
                * ic_loss
            )

        return total, {
            "data": data_loss,
            "pde": pde_loss,
            "bc": bc_loss,
            "ic": ic_loss
        }
```

18 Example: Burgers Residual

Consider

$$u_t + uu_x - \nu u_{xx} = 0. \tag{55}$$

A residual function can be implemented as follows.

```

def burgers_residual(
    u,
    x,
    t,
    viscosity
):

    u_t = torch.autograd.grad(
        u,
        t,
        grad_outputs=torch.ones_like(u),
        create_graph=True
    )[0]

    u_x = torch.autograd.grad(
        u,
        x,
        grad_outputs=torch.ones_like(u),
        create_graph=True
    )[0]

    u_xx = torch.autograd.grad(
        u_x,
        x,
        grad_outputs=torch.ones_like(u_x),
        create_graph=True
    )[0]
```

```

    residual = (
        u_t
        + u * u_x
        - viscosity * u_xx
    )

    return residual
```

The physics loss is

```

residual = burgers_residual(
    prediction,
    x,
    t,
    viscosity
)

physics_loss = torch.mean(
    residual ** 2
)
```

19 Generic PINO Training Loop

```

optimizer = torch.optim.Adam(
    model.parameters(),
    lr=1e-3
)

for epoch in range(1000):

    model.train()

    optimizer.zero_grad()

    # Neural operator prediction
    prediction = model(
        model_input
    )

    # Data loss
    data_loss = torch.mean(
        (prediction - target) ** 2
    )

    # PDE residual
    residual = physics_residual(
        prediction,
        coordinates,
        parameters
    )

    # Physics loss
    physics_loss = torch.mean(
        residual ** 2
```

```
    )

    # Total PINO loss
    loss = (
        lambda_data * data_loss
        +
        lambda_pde * physics_loss
    )

    loss.backward()

    optimizer.step()

    if epoch % 100 == 0:

        print(
            f"Epoch {epoch} | "
            f"Total: {loss.item():.6e} | "
            f"Data: {data_loss.item():.6e} | "
            f"PDE: {physics_loss.item():.6e}"
        )
```

20 Residual Evaluation on a Grid

For a discretized PDE,

$$X = \{x_i\}_{i=1}^{N}, \tag{56}$$

the neural operator generates

$$\hat{U} = \mathcal{G}_\theta(A). \tag{57}$$

The discrete PDE residual can be written as

$$R_\theta = \mathcal{F}_h(\hat{U}; A), \tag{58}$$

where $\mathcal{F}_h$ is a numerical discretization of the PDE operator.
The physics loss becomes

$$\boxed{\mathcal{L}_{\text{physics}} = \frac{1}{N}\|R_\theta\|_2^2.} \tag{59}$$

This is particularly useful when the neural operator works directly with discretized fields.

21 Important Distinction from a Standard PINN

A standard Physics-Informed Neural Network represents a solution using

$$u_\theta(x, t). \tag{60}$$

It learns a single solution function for a specified PDE instance.
A neural operator represents

$$\mathcal{G}_\theta : a \mapsto u. \tag{61}$$

It learns a mapping across a family of PDE instances.

PINO therefore combines the two ideas:

$$\boxed{\text{PINN} = \text{Neural Function} + \text{Physics Loss}} \tag{62}$$

whereas

$$\boxed{\text{PINO} = \text{Neural Operator} + \text{Physics Loss.}} \tag{63}$$

22 General Recipe

Given an arbitrary neural operator

$$\mathcal{G}_\theta, \tag{64}$$

the conversion to a PINO can be summarized as

$$\boxed{\begin{array}{ll} 1. & \hat{u} = \mathcal{G}_\theta(a), \\ 2. & r_\theta = \mathcal{F}[\hat{u}; a], \\ 3. & \mathcal{L}_{\text{PDE}} = \|r_\theta\|^2, \\ 4. & \mathcal{L} = \lambda_d \mathcal{L}_{\text{data}} + \lambda_p \mathcal{L}_{\text{PDE}} + \lambda_b \mathcal{L}_{\text{BC}} + \lambda_i \mathcal{L}_{\text{IC}}, \\ 5. & \theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}. \end{array}} \tag{65}$$

Therefore, if

$$\mathcal{G}_\theta = \text{FNO}_\theta, \tag{66}$$

one obtains PINO-FNO.
If

$$\mathcal{G}_\theta = \text{DeepONet}_\theta, \tag{67}$$

one obtains PINO-DeepONet.
If

$$\mathcal{G}_\theta = \text{Transolver}_\theta, \tag{68}$$

one obtains PINO-Transolver.
The same construction applies to any differentiable neural operator.

23 Conclusion

The essential idea of Physics-Informed Neural Operators is to constrain an operator-learning model using the governing physical equations.

The neural operator provides

$$a \xrightarrow{\mathcal{G}_\theta} \hat{u}, \tag{69}$$

while the PDE provides

$$\hat{u} \xrightarrow{\mathcal{F}} r_\theta. \tag{70}$$

Training minimizes both prediction error and physical inconsistency:

$$\boxed{\mathcal{L}_{\text{PINO}} = \lambda_d \mathcal{L}_{\text{data}} + \lambda_p \mathcal{L}_{\text{physics}} + \lambda_b \mathcal{L}_{\text{BC}} + \lambda_i \mathcal{L}_{\text{IC}}.} \tag{71}$$

The defining principle is therefore

$$\boxed{\text{PINO} = \text{Neural Operator} + \text{PDE Residual Constraint.}} \tag{72}$$